

JavaScript

El llenguatge de programació



Javascript

Va ser desenvolupat per Netscape i Sun Microsystems, i es pot fer servir en els clients Web de Netscape (a partir de la versió 2.0) i Microsoft (a partir de la versió 3.0); hi ha lleugeres diferències en els intèrprets JavaScript de cada plataforma.

Javascript

- Llenguatge de programació interpretat, és a dir, que no requereix compilació, utilitzat principalment en pàgines web, amb una sintaxi semblant a la del llenguatge Java i el llenguatge C.
- Llenguatge de programació que ens permet donar-li interactivitat al lloc web. Treballa del costat del navegador.

Javascript és un llenguatge que pot ser utilitzat per professionals i per als qui s'inicien en el desenvolupament i disseny de llocs web. No requereix de compilació ja que el llenguatge funciona del costat del client, els navegadors són els encarregats d'interpretar aquests codis.

Javascript té l'avantatge de ser incorporat a qualsevol pàgina web, pot ser executat sense la necessitat d'instal·lar un altre programa per ser visualitzat

CARACTERÍSTIQUES

Java per la seva banda té com a principal característica ser un llenguatge independent de la plataforma.

Es pot crear tot tipus de programa que pot ser executat en qualsevol ordinador del mercat: Linux, Windows, Apple, etc.

A causa de les seves característiques també és molt utilitzat per a internet.

On puc veure funcionant Javascript?

Entre els diferents serveis que es troben realitzats amb Javascript a Internet es troben:

- Correu
- Xat
- Cercadors d' Informació

També podem trobar o crear codis per inserir-los a les pàgines com:

- Rellotge
- Comptadors de visites
- Dates
- Calculadores
- Validadors de formularis
- Detectors de navegadors i idiomes

Com IDENTIFICAR CODI JAVASCRIPT?

- El codi javascript podem trobar-lo dins de les etiquetes `<body></body>` de les nostres pàgines web.
- En general s'insereixen entre:
`<script></script>`.
- També poden estar ubicats en fitxers externs usant:

`<script type="text/javascript" src="elmeucodi.js"></script>`

ÉS COMPATIBLE AMB ALTRES NAVEGADORS

Javascript és suportat per la majoria dels navegadors com Internet Explorer, Netscape, Opera, Mozilla Firefox, entre d'altres.

Inserir JavaScript en el nostre projecte

Pot ser afegit de dues formes:

Interna: Mitjançant l'etiqueta `<script>`

```
<script>
  function onClick(){
    document.getElementById("example").innerHTML = "Example function";
  }
</script>
```

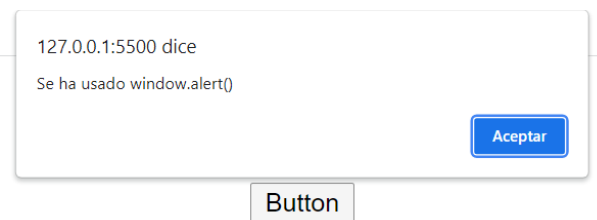
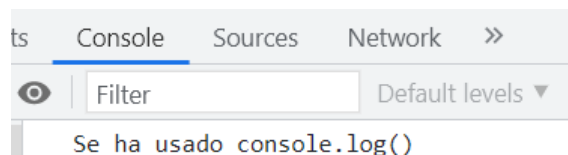
Externa: Mitjançant l'etiqueta `script` cridant a un arxiu extern especificant la seva ruta amb l'atribut `"src"`. Té l'avantatge que separa l'HTML del JS, fent que siguin més fàcils de llegir i mantenir.

```
<script src="main.js"></script>
```

Opcions per a la sortida de dades en JS

JavaScript pot mostrar dades de formes diferents:

- Mitjançant la consola del navegador utilitzant **console.log()**.
- Escrivint en una caixa d'alertes, usant **window.alert()**.
- Escrivint en un element HTML, utilitzant **innerHTML**. Per accedir a un element podem fer servir el mètode `element = document.getElementById(id)`.
- Escrivint a la sortida estàndard d' HTML, mitjançant **document.write()**. Això és només recomanable en fases de prova.



Sintaxis de JavaScript

La sintaxi de JavaScript defineix com el codi dels programes està construït. Ens trobem amb el següent:

- **Valors:** Poden ser literals, constants o variables, que emmagatzemen els valors de les dades.
- **Operadors:** Hi ha aritmètics, de comparació i lògics.
- **Expressions:** Combinen operadors, valors literals i variables; que donen com a resultat un valor.
- **Paraules clau:** Identifiquen una acció a realitzar, com pot ser crear una variable.
- **Comentaris:** Part del codi que no ha de ser executat i serveixen per a la comprensió del codi.
- **Identificadors:** Són noms que serveixen per identificar variables, funcions o paraules clau.

Exemple de programa Javascript.

```
<HTML>
<HEAD>
<TITLE>My First Script</TITLE>
</HEAD>

<BODY>
<H1>Let's Script...</H1>
<HR>
<SCRIPT LANGUAGE="JavaScript">
<!-- hide from old browsers
document.write("This browser is version " + navigator.appVersion)
document.write(" of <B>" + navigator.appName + "</B>.")
// end script hiding -->
</SCRIPT>
</BODY>
</HTML>
```

> Etiqueta <script>

S' utilitza per inserir o fer referència a un script executable.

Aquest es pot incloure directament dins l' etiqueta o amb un enllaç extern.

```
<script src="scripts.js"></script>
```

> Etiqueta <script>

```
<script src="scripts.js" async></script>
```

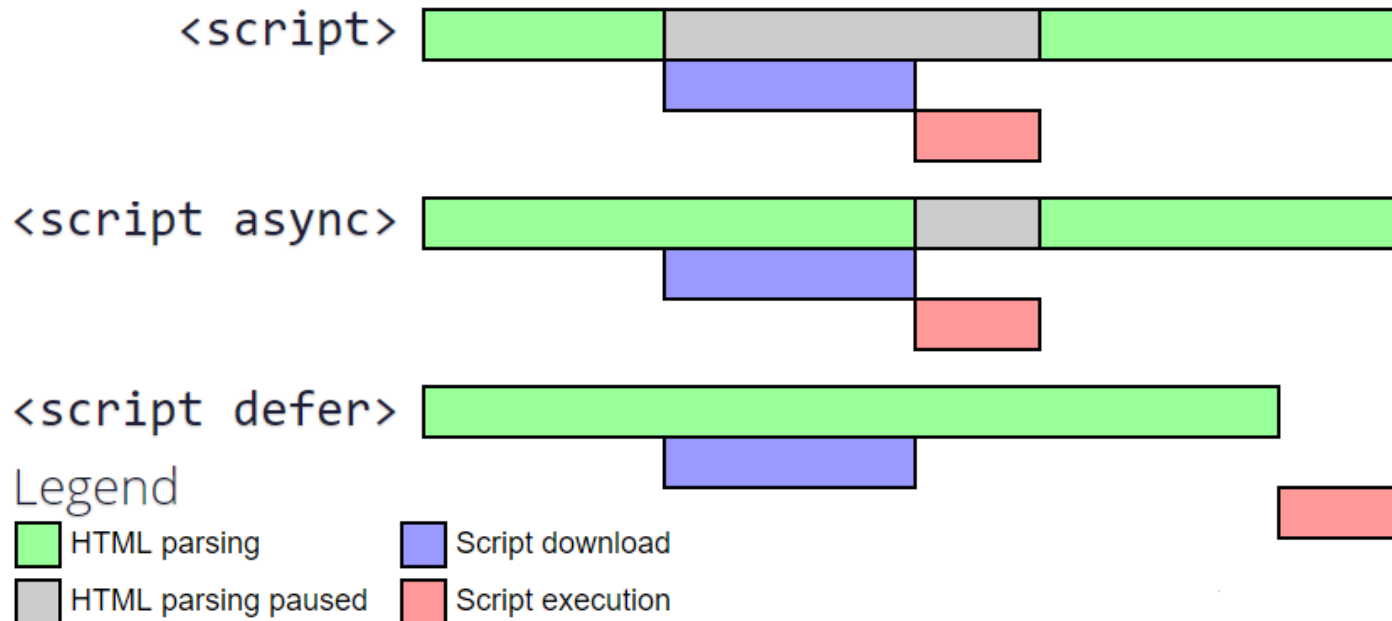
Indica que l'script s'ha de descarregar de manera asíncrona i després executar-se tan aviat com estigui disponible.

```
<script src="scripts.js" defer></script>
```

L' script s' ha de descarregar de manera asíncrona i després executar-se un cop s' hagi descarregat per complet i s' hagi descarregat el document HTML per complet.

> Etiqueta <script>

async vs defer



COMENTARIS

- Són parts del programa que l'interpret no llegeix i que, per tant, serveixen per explicar i aclarir.
- Van **compresos entre barres i bifocades** com en l'exemple següent:
- /*comentari*/
- El comentari pot constar de més d' una línia o d' una de sola, mentre que l' interpret no accepta el comentari imbricat.

Elaboració de comentaris per a funcions i programes en extens.

Dos tipus de comentaris permissos

- Comentaris en línia que comencen amb una doble barra: //,
- Comentaris multilínia, que comencen amb /* i acaben amb */.
- **No poden** inserir-se fora de les marques que s'han de fer, ja que, en cas contrari, HTML els considera part del text

COMENTARIS EN JAVA SCRIPT

```
<html>
<head>
<title>Portal web - aprendraprogramar.com</title> <meta charset="utf-8">
<script type="text/javascript">
/* Funcions JavaScript
Versió: 0.1
Autor: Jordi Giner
Curs: Tutorial bàsic del programador web: JavaScript des de zero
*/

//Funció que mostra missatge de benvinguda
function mostrarMissatge1() {
alert('Benvingut al curs JavaScript de Grup CIEF');
}

function mostrarMissatge2() {
//Mensaje si se hace click sobre párrafo
alert('Ha fet click sobre el paràgraf inferior');
}
</script>
</head>
<body>
<div>
<p>Aquí un paràgraf de text situat abans de la imatge, dins d' un div contenidor</p>

<p style="background-color:yellow;" onclick="mostrarMissatge2()">Aquí un altre paràgraf de text. JavaScript és
un llenguatge utilitzat per dotar d'efectes dinàmics les pàgines web.
</p>
</div>
</body>
</html>
```

Espais en blanc

- A Javascript **no li interessien els espais en blanc.**
- Excepte els que es troben a les cadenes, per la qual cosa es poden ometre o també augmentar.
- El seu ús, de tota manera, augmenta la llegibilitat del programa pel que aconsellem vivament el seu ús

COMETES

- Un element important són les **cometes**, tant **simples (' ')**, com **dobles (" ")**.
- Les **cometes dobles** es fan servir per **tancar parts de codi Javascript**, i, juntament amb les **simples**, **per tancar també les cadenes** (seqüències de caràcters).
- Per això, cal parar atenció a nidificar dues cadenes compreses entre cometes simples, així com a usar les cometes dobles per a les cadenes si aquestes serveixen ja per tancar codi Javascript.

Variables

Variables i constants

- Les variables són contenidors per a valors que s'assignen a les dades, i que poden canviar durant l'execució del programa.
- Una variable és un espai en la memòria de la computadora que ens permet emmagatzemar informació per després recuperar-la i utilitzar-la.
- Les constants són contenidors de valors que no es modifiquen en cap moment.
- Aquests dos tipus de contenidors han de ser identificats amb noms únics. Se'ls assigna un valor mitjançant el signe "=", podent ser igualat el seu valor a una altra variable, a un valor o a una expressió.
- Les variables es declaren mitjançant les paraules **var** (es pot declarar més d'una vegada) i **let** (si només es declararà una vegada). Després de ser declarada, una variable no té valor (**undefined**), i aquest ha de ser assignat.
- Les constants són declarades mitjançant la paraula **const**, i el seu valor no ha de ser reassignat un cop es declara.

```
let a;
```

```
const C = a + b;
```

```
var b = 5;
```

> Variables > sintaxis

- ❑ Hi ha diverses formes de declarar una variable (var, let, const)
- ❑ Per ara, utilitzarem la paraula let.
- ❑ L' **igual simple** determina el valor de la variable.
- ❑ Per al nom de la variable es recomana l'ús de **camelCase**.

```
let miVariable = VALOR;
```


> Variables > sintaxis

- ❑ En els següents exemples podem veure que no totes les variables són iguals.
- ❑ Aquestes diferències són els diferents tipus de dades que maneja JavaScript.

```
let nombre = "Leia";
```

```
let apellido = "Organa";
```

```
let edad = 32;
```

```
let sabeJavascript = false;
```

Variables.

Es tenen quatre tipus bàsics:

- Nombres (enters, decimals, etc...)
- Lletres i números (cadenes de caràcters)
- Valors lògics (true i false)
- Objectes (una finestra, un text, un formulari, el navegador, l'historial, etc...)

Variables

Definició de variables a JavaScript

- La forma general per declarar una variable és la següent:

var nom_variable

nom_variable = "valor"

Variables

Exemples de definició de variables:

```
var miVar = 1234;
```

```
var miVar2 = 12.34;
```

```
var miCadena = 'Hola, món';
```

```
var matriu = new Array();
```

```
var matriu2 = new Array(15);
```

Variables locals i globals

```
var variableGlobal=0;
```

```
function novaFuncio() {  
  var variableLocal=1;  
  variableGlobal =0;  
}
```

Variables

No es declara el tipus de dada de la variable, el tipus s'assigna en execució i pot canviar de tipus durant la seva vida, per exemple:

laMevaVariable=4;

laMevaVariable="Una_Cadena";

Operadors aritmètics

Podem realitzar les operacions clàssiques mitjançant els signes `*`, `+`, `-`, `**`, `/`, `%`, combinant-les en expressions amb variables, constants o nombres.

Mitjançant l'operador `++` incrementarem en una unitat el valor d'una dada, i farem el mateix per restar una unitat amb `--`.

Podem veure en aquesta taula resum altres operadors disponibles:

Operador	Exemple	Equivalent a
<code>=</code>	<code>x = y</code>	<code>x = y</code>
<code>+=</code>	<code>x += y</code>	<code>x = x + y</code>
<code>-=</code>	<code>x -= y</code>	<code>x = x - y</code>
<code>*=</code>	<code>x *= y</code>	<code>x = x * y</code>
<code>/=</code>	<code>x /= y</code>	<code>x = x / y</code>
<code>%=</code>	<code>x %= y</code>	<code>x = x % y</code>
<code>**=</code>	<code>x **= y</code>	<code>x = x ** y</code>

Exemples d' operacions

```
let a = 3; // La variable a se declara con valor 3
let b;    // Se declara la variable b
b = a + 4; // b = a + 4 = 7
b = b++;  // b se incrementa en 1, vale 8
var c = b; // Se declara c con valor 8
c /= 2;    // c = c/2 = 4
const d = 5 ** 2; // Se declara la constante d = 25
c %= a;    // c = c mod a = 1
a -= 1;    // a = a - 1 = 3
let m = (a + b + c + d)/4 // m es la media de todas, m = 8,625
```


Operadors de comparació i lògics

Les comparacions i les operacions lògiques retornen un valor booleà segons si una condició es compleix o no.

Operadors de comparació

Operador	Significat
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

Operadors lògics

Operador	Ús	Retorna true si...
AND (&&)	expr1 && expr2	Les dues expressions són certes
OR ()	expr1 expr2	Almenys una de les dues expressions és certa
NOT (!)	!expr	L'expressió és falsa

Operadors.

Operadores aritméticos:

Operador	Significado	Ejemplo
+	suma	números y cadenas
-	resta	
*	producto	
/	división	
%	módulo (resto)	20% 10 (= 0)
++	suma tipográfica	variable++; ++variable; (variable = variable + 1)
-- (dos guiones)	resta tipográfica	variable; --variable; (variable = variable - 1)

Operadores de asignación

Operadors

Operadores de asignación:

Operador	Significado	Ejemplo	Es igual a
=	Asignación de datos	<code>x = 1;</code>	
<code>+=</code>	Asignación y suma	<code>max += 1;</code>	<code>x = x + 1;</code>
<code>-=</code>	Asignación y resta	<code>x -= 1;</code>	<code>x = x - 1;</code>
<code>*=</code>	Asignación y producto	<code>x *= 1;</code>	<code>x = x * 1;</code>
<code>/=</code>	Asignación y división	<code>x /= 1;</code>	<code>x = x / 1;</code>
<code>%=</code>	Asignación y módulo	<code>x %= 1;</code>	<code>x = x % 1;</code>

Operadors

Operadores condicionales (comparativos):

Operador	Significado	Ejemplo
==	es igual a	5 == 8 es falso
!=	no es igual a	5 != 1 es verdad
>	es mayor que	5 > 1 es verdad
<	es menor que	5 < 8 es verdad
>=	es mayor o igual que	5 >= 8 es falso
<=	es menor o igual que	5 <= 1 es falso

Operadors

Operadores lógicos:

Operador	Significado	Ejemplo
&&	Y	1 == 1 && 2 < 1 es falso
	O	1 == 2 15 > 2 es verdad
!	NO	!(1 > 2) es verdad

Operadors

Table 40-10 JavaScript Operator Precedence

Precedence Level	Operator	Notes
1	()	From innermost to outermost
	[]	Array index value
	function()	Any remote function call
2	!	Boolean Not
	~	Bitwise Not
	-	Negation
	++	Increment
	--	Decrement
	new	
	typeof	
	void	
	delete	Delete array or object entry
3	*	Multiplication
	/	Division
	%	Modulo
4	+	Addition
	-	Subtraction
5	<<	Bitwise shifts
	>	
	>>	
6	<	Comparison operators

6

<

Comparison operators

<=

>

>=

7

==

Equality

!=

8

&

Bitwise And

9

^

Bitwise XOR

10

|

Bitwise Or

<i>Precedence Level</i>	<i>Operator</i>	<i>Notes</i>
11	&&	Boolean And
12		Boolean Or
13	?	Conditional expression
14	=	Assignment operators
	+=	
	-=	
	*=	
	/=	
	%=	
	<<=	
	>=	
	>>=	
	&=	
	^=	
	=	
	,	
15	,	Comma (parameter delimiter)

Tipus de dades

Dades bàsiques

Tipus de dades

El concepte de tipus de dades és molt important en programació. Ens donen informació respecte als valors de les variables que estem utilitzant, i dictaminen de quina manera podem utilitzar-los.

Les variables en JS poden canviar de tipus de forma dinàmica, és a dir, es pot reutilitzar la mateixa variable per allotjar un altre tipus de dades. Els més importants són:

- **Strings:** Contenen cadenes de caràcters entre cometes.

```
let name = "Juanito";
```

- **Numbers:** Contenen nombres, tant enters com decimals.

```
let age = 35;
```

- **Booleanos:** Poden prendre dos valors, true o false.

```
let isMale = true;
```

Tipus de dades

- **Arrays:** Contenen diferents elements, cadascun amb un índex començant pel 0.

```
let grandparents = ["Pepe", "Manolo", "Maria", "Laura"];
```

- **Objectes:** Tenen diferents propietats que es representen mitjançant parells nom-valor, separades per comes.

```
let juanito = { name:"Juanito", age: 35, isMale: true, grandparents: ["Pepe", "Manolo", "Maria", "Laura"] }
```

- **Undefined:** Representa una dada sense valor. És el que prenen les variables en ser declarades.

Podem obtenir el tipus d'una dada mitjançant l'operador **typeof**.

```
typeof age; // Nos dará como resultado el tipo number
```

> Dades > números

Números

- ❑ Els tipus numèrics s' escriuen directament sense cap aclariment particular.
- ❑ Són vàlids tant números positius, negatius i decimals.

```
let edad = 52;
```

```
let precio = 180.8;
```

```
let latitud = -33.72;
```

> Dades > strings

Strings

- ❑ Són textos.
- ❑ Es defineixen entre cometes (simples o dobles però hem d'utilitzar-la per obrir i tancar)

```
let nombre = "Leia";
```

```
let apellido = 'Organa';
```

> Dades > booleanos

Booleanos

- ❑ Són valors de veritable o fals.
- ❑ Es defineixen simplement com a TRUE o FALSE.

Tip: utilitzar noms representatius del valor de la variable.

```
let esMayorEdad = true;
```

```
let esHomePage = false;
```

> Dades > booleanos

Booleanos

- ❑ Són valors de veritable o fals.
- ❑ Es defineixen simplement com a TRUE o FALSE.

Tip: utilitzar noms representatius del valor de la variable.

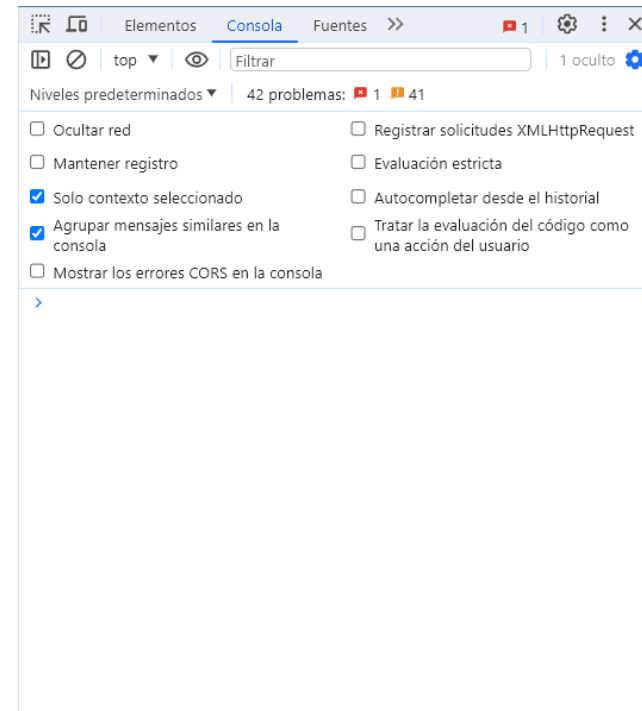
```
let esMayorEdad = true;
```

```
let esHomePage = false;
```

Consola JS

> Consola JS

La consola de JavaScript és una eina que es troba en la majoria dels navegadors web i permet als desenvolupadors interactuar amb el codi JavaScript i depurar la seva aplicació.



> Consola JS > console.log()

Per imprimir les variables a la consola i conèixer el seu valor utilitzem una funció ja inclosa en JS anomenada console.log.

```
let planeta = "Urano";
```

```
console.log(planeta);
```

```
> let planeta = "Urano";  
   console.log(planeta)
```

```
Urano
```

Exercici

Practicar JS

> Exercici

Pas previ: crear un arxiu JS i linkejar-lo al teu index.html.

1. Defineix en un arxiu JS variables amb:
 - a. El teu nom
 - b. El teu cognom
 - c. La teva edat
 - d. Si tens mascota
2. Després imprimeix les variables a la consola.
3. Saps de quin tipus és cada variable?

Tipus de dades

Arrays

> Dades > arrays

És un objecte especial que permet crear grups ordenats de dades i ens proveeix eines per treballar-hi.

Els arrays poden agrupar qualsevol tipus de dades.

```
let planetas = ["Mercurio", "Jupiter", "Saturno"];
```

```
let tieneAnillos = [false, true, true];
```

> Dades > arrays

- ❑ Un array es defineix utilitzant "[]" al voltant dels elements que contingui.
- ❑ Es pot definir un array sense elements utilitzant simplement "[]".
- ❑ O pot contenir diferents tipus de dades.

```
let arrayVacio = [];
```

```
let arrayConElementos = [23,  
11100, 7894455778];
```

```
let mascota = ["Garfield",  
"naranja", true, 1978];
```

> Dades > arrays

També hi ha arrays d'arrays!

```
let planetas = [  
    ["Mercurio", false],  
    ["Jupiter", true],  
    ["Saturno", true]  
];
```


> Dades > arrays

Llegir els arrays

- ❑ Els arrays asseguren un ordre de les dades que contenen.
- ❑ En javascript el primer element està en l'índex 0 (zero).
- ❑ Per especificar l'índex de l'arrelament l'especifiquem entre "[]" immediatament després del nom de la variable.

```
let planetas = [  
    "Mercurio",  
    "Jupiter",  
    "Saturno"];
```

```
console.log(planetas[1]);
```

> Dades > arrays

Llegir els arrays

- ❑ Els arrays asseguren un ordre de les dades que contenen.
- ❑ En javascript el primer element està en l'índex 0 (zero).
- ❑ Per especificar l'índex de l'arrelament l'especifiquem entre "[]" immediatament després del nom de la variable.

```
let planetas = [  
    "Mercurio",  
    "Jupiter",  
    "Saturno"];
```

```
console.log(planetas[1]);
```

> Dades > arrays

Length

- ❑ De vegades necessitem saber que tants elements té un arrelament i la propietat **length** proveeix específicament aquest valor.

```
let planetas = [  
    "Mercurio",  
    "Jupiter",  
    "Saturno"];
```

```
> console.log(planetas.length);  
>
```

Tipus de dades

Objectes literals

> Dades > objectes literals

Són un altre tipus de variable que permet guardar molta informació en una sola variable, en principi, similars a l'array.

Són elements que tenen una sèrie de característiques o propietats que els defineixen.

> Dades > objectes literals

- ❑ Agrupen les seves característiques entre claus{}
- ❑ A diferència dels arrays cada element s'identifica mitjançant el seu nom d'atribut (en comptes de la seva posició).
- ❑ Fem servir els dos punts per separar el nom del valor.
- ❑ S'hi accedeix amb el nom de la propietat.

```
let planeta = {  
    nombre: "Urano",  
    anillos: true,  
    distanciaSol: 20,  
    lunas: [ "Titania", "Oberón",  
            "Umbriel", "Ariel", "Miranda"]  
};
```

```
> console.log(planeta.lunas[0]);  
> "Titania"
```

Funcions

Les funcions són blocs de codi que executen una tasca determinada i que poden ser executats només quan se'ls anomena mitjançant una part del codi en execució, per exemple davant d'un esdeveniment.

Les funcions es defineixen mitjançant la paraula clau **function**, seguida d'un identificador, paràmetres d'entrada entre parèntesis i un bloc de codi a executar entre { }.

En invocar una funció, se li passen com a argument valors que seran tractats com a paràmetres dins d'ella.

Quan volem acabar l'execució una funció, fem servir la paraula **return**, i tornarem a la part del codi on va ser invocada. Les funcions poden retornar o no un valor.

Utilitzem funcions per poder reutilitzar codi, es defineix una vegada i es fa servir totes les que es necessiti. També poden ser utilitzades per assignar valors.

Funcions

Les variables que són declarades en una funció es denominen variables locals, i només poden ser accedides dins de la funció.

Crida a funcions

```
let result = sum(a, 7);  
alertBox("This is a example");  
let c = addFive(b);
```

Funcions

```
function sum (x, y) {  
    return x + y;  
}  
  
function alertBox (text) {  
    window.alert(text);  
}  
  
function addFive (x) {  
    let y = x + 5;  
    return y;  
}
```


String i els seus mètodes

Ja hem vist què són els **strings**, vam veure alguns dels mètodes i propietats que tenen:

- **length**: Propietat per saber la longitud d' un string.
- **substring (inici, fi)**: Mètode amb el qual podem treure una subcadena entre dues posicions de l'string.
- **toUpperCase ()**: Per convertir els seus caràcters a majúscules.
- **concat (string, string,...)**: Aquest mètode retorna la unió de diversos strings.
- **split (string)**: Mètode que ens permet convertir l' string en un arrelament amb les subcadenaes que queden separades pel caràcter que li especifiquem.
- **indexOf (string)**: podem saber quina és la posició d'un caràcter o un string a la cadena.
- **slice (inici, fi)**: Per extreure la subcadena continguda entre dues posicions.
- **replace (string, string)**: Permet reblar una subcadena per una altra.
- **includes (string)**: Retorna un valor booleà segons si la nostra cadena conté la que li passem.

Array i els seus mètodes

Ja hem vist el que són els **arrays**, anem a veure certs mètodes propis que té:

- **pop ()**: Esborra l'últim element d'un arrelament.
- **push (element)**: Afegeix un nou element al final d' un array,
- **sort ()**: Aquest mètode ordena un arrelament de forma alfabètica.
- **reverse ()**: Inverteix l' ordre dels elements d' unarray.
- **forEach (function)**: Crida a una funció per a cada element de l'array.
- **splice (índex, número d'elements, elements a insertar)**: Esborra tants elements com li especifiquem, a més de ser capaç d'afegir elements nous en el seu lloc.

10	34	15	20
v[0]	v[1]	v[2]	v[3]



Mètodes en els tipus de dades

En els números, podem fer servir el mètode **toExponential()**, que el converteix a notació exponencial; el mètode **toFixed(number)**, que ens forma el nombre amb la quantitat de dígitos decimals que li especifiquem; i el mètode **toPrecision(number)**, que el nombre amb la longitud que li passem com a paràmetre.

Tant per a arrays, booleans, numbers i altres tipus de dades podem utilitzar el mètode **toString()**, que converteix els nostres valors a cadenes de caràcters.

El mètode **parseInt(string, base)** ens retorna un sencer en base 10 a partir d'un número o string, definint nosaltres la base en la qual hi ha el primer argument.

A més de les que estan ja implementades en JS, serem nosaltres mateixos els que augmentarem aquest nombre de mètodes mitjançant funcions, sobretot en els objectes que creiem des de zero.

Objectes en JS

Els objectes són tipus de dades que poden contenir diversos atributs i mètodes propis. Aquests atributs o propietats s'escriuen amb parells nom:valor, i poden ser accedits des de qualsevol part mitjançant `objectName.propertyName` o `objectName["propertyName"]`.

Els mètodes són accions que poden ser realitzades en els objectes, en són funcions pròpies. Podem accedir-hi de forma similar als atributs amb la forma d'actuació..

Mitjançant la paraula clau **this**, podem fer referència a l' objecte en qüestió dins dels seus propis mètodes. Podem definir les seves propietats tant de forma conjunta com d'una en una.

```
let car = new Object();  
car.model = 'Ford Focus';  
car.km = 120000;  
car.registration = '0123ABC';
```

```
let car = {  
  model: 'Ford Focus',  
  km: 120000,  
  registration: '0123ABC',  
  getData: function getData(){ // Devuelve Ford Focus 120000km 0123ABC  
    return this.model + ' ' + this.km + 'km ' + this.registration;  
  }  
}
```

Crear objectes en JS

Ja hem vist com definir els objectes, ara vam aprendre a crear-los mitjançant una funció constructora, que ens permetrà crear diversos objectes diferents amb les mateixes propietats. Ho farem de la següent manera:

```
function Car(model, km, registration){  
  this.model = model;  
  this.km = km;  
  this.registration = registration;  
}  
  
let focus = new Car('Ford Focus', 120000, '0123ABC');  
let sandero = new Car('Dacia Sandero', 310000, '0321ADC');
```

Què és un set?

Un **set** és un tipus de dada que representa una col·lecció iterable de valors únics. Cadascú pot aparèixer només una vegada al set.

Té diversos mètodes propis, com `add( )`, `forEach( )`, `values( )`, `has( )`, `size`,...

```
let setExample = new Set(); // Declaramos un set
setExample.add(5); // Añadimos un elemento de tipo number
setExample.add('cadena'); // Añadimos un elemento de tipo string
let person = {name: "Ramon", age: "20"};
setExample.add(person); // Añadimos un elemento de tipo objeto
console.log(setExample.has('cadena')); // Imprime true
console.log(setExample); // Imprime [5, 'cadena', {name: "Ramon", age: "20"}]
console.log(setExample.size); // Imprime 3
console.log(setExample.has("Ramon")); // Imprime false
setExample.delete(5); // Eliminamos el 5
setExample.add(person); // No añade nada
console.log(setExample); // Imprime ['cadena', {name: "Ramon", age: "20"}]
setExample.clear(); // Eliminamos todos los elementos
```

Què és un map?

Un **map** és un tipus de dada que representa una col·lecció de parells clau-valor on les claus poden ser qualsevol tipus de dada, a diferència dels objectes en què han de ser strings o símbols...

Té diversos mètodes propis, com `set()`, `get()`, `delet()`, `has()`, `clear()`,...

```
let mapExample = new Map(); // Declaramos un map
let february = {month: 'February'}; // Declaramos un objeto
mapExample.set('January', 31); // <string, number>
mapExample.set(february, '28'); // <object, string>
mapExample.set(3, 31); // <number, number>
mapExample.set('April', 30); // <string, number>
console.log(mapExample.has(3)); // Imprime true
console.log(mapExample.get(february)); // Imprime 28
mapExample.set('January', 32); // January pasa a tener el valor 32
mapExample.delete('April'); // Borra 'April': 30
console.log(mapExample.size); // Imprime 3
mapExample.clear(); // Elimina todos los elementos
```

Sentència “If-else”

La sentència **if** és una estructura de control condicional que s'utilitza per determinar la realització d'una acció si una certa condició es compleix. Té la següent sintaxi:

- La paraula clau **if**, seguida de la condició i un bloc de codi a executar.
- La sentència opcional **else** serveix per executar una acció si la condició és falsa.
- La sentència opcional **else if** s'utilitza per especificar una nova condició si l'anterior ha estat falsa.

```
if (condicion) {  
    // bloque de código si se cumple la condición  
} else {  
    // bloque si no se cumple la condición  
}
```

```
if (condicion1) {  
    // bloque de código si se cumple la condición 1  
} else if (condicion2) {  
    // bloque si se cumple la condición 2  
}
```


Sentència “switch-case”

La sentència **switch** és una estructura de control que es fa servir per determinar una acció a realitzar basada en les diferents possibilitats que pot prendre una variable o expressió. Té la següent sintaxi:

- La paraula clau **case** defineix els valors que pot prendre la variable o expressió que decideix quin cas s'executa.
- La paraula **break** até l'execució del bloc de codi de switch.
- Fem servir **default** si no es compleix cap condició, i en aquest cas executa el seu codi.

```
switch (variable) {  
    case value1:  
        // Bloque de código  
        break;  
    case value2:  
        // Bloque de código  
        break;  
    default:  
        // Bloque de código  
        break;  
}
```

Què són els bucles?

Els bucles s'utilitzen per executar un bloc de codi un nombre determinat de vegades. Poden ser utilitzats per recórrer el mateix codi amb diferents valors cada vegada, per recórrer arrays i per a altres moltes funcions.

Els bucles ens permeten executar la mateixa tasca una i altra vegada, el que s'anomena **iteració**. Un bucle normalment té aquestes característiques:

- Un comptador: Que s'inicia a un determinat valor i anirà canviant durant les iteracions.
- Una condició de sortida: És la que marca el final del bucle, pot ser que el comptador arribi a determinat valor, la consecució d'un valor que buscàvem,...
- Un iterador: Incrementa el valor del comptador fins a assolir una condició de sortida.

Aquests bucles van a ajudar-nos d'una forma senzilla a realitzar tasques repetitives en poques línies. Els més famosos són el bucle **for** i el bucle **while**.

Bucle “for”

El bucle **for** serveix per executar un bloc de codi un nombre determinat de vegades. Aquest bucle té la següent sintaxi:

- La paraula clau **for**, tres declaracions entre parèntesis i un bloc de codi que s'executarà una o més vegades.
- La primera declaració és executada abans d' entrar al bucle, la segona defineix la condició per tornar a executar el bloc de codi posterior i la tercera és executada en acabar cada iteració.

```
for (statement1; statement2; statement3) {  
    //Bloque de código a ejecutar en cada iteración  
}
```

```
let sumaVector = 0;  
let v = [1,2,3,4]  
for (i = 0; i < v.length; i++) {  
    sumaVector += v[i];  
}
```

Bucles “for..in” i “for..of”

El bucle **for... in** es fa servir per iterar sobre les propietats d' un objecte.

```
for (variable in objeto){  
    // Bloque a ejecutar  
}
```

```
let charIndex = {a: 1, b: 2, c: 3}  
for (x in charIndex){  
    console.log(x);  
    console.log(charIndex[x]);  
}
```

El bucle **for... of** s'utilitza per iterar sobre un objecte iterable, com poden ser arrays, strings, llistes de nodes,...

```
for (variable of iterable){  
    // Bloque a ejecutar  
}
```

```
for (char of 'cadena'){  
    console.log(char)  
}
```

Bucle “while” i “do while”

S' utilitzen per executar un bloc de codi una i altra vegada mentre la condició del bucle sigui certa. Tenen la següent sintaxi:

- La paraula clau **while**, seguida d'una condició que en cas de ser veritable fa que s'executi el bucle de nou. Pot anar seguida d'un bloc de codi entre { }. Hem de tenir cura per no entrar en un bucle infinit.
- La paraula **do**, que especifica el bloc de codi a executar.

```
while(condicion){  
  // Bloque a ejecutar mientras se cumpla la condicion  
}
```

```
do {  
  // Bloque a ejecutar mientras se cumpla la condicion  
}  
while(condicion);
```

```
let i = 0;  
let sumaVector = 0;  
let v = [1,2,3,4]  
while(i < v.length){  
  sumaVector += v[i];  
  i++;  
}
```

Bucles i sentències imbricades

Quan implementem codi en JS, podem anellar diverses sentències i bucles, la qual cosa ens permetrà realitzar estructures de control més precises, recórrer diversos objectes alhora, accedir a dades dins d'iterables amb més d'un índex, com poden ser arrelats dins d'altres arrays,... entre altres moltes possibilitats.

```
function printOdd(a){
  if (typeof(a) == "number"){
    if (a%2==0){
      console.log('Falso, la variable es par');
    }
    else{
      console.log('Verdadero, la variable es impar');
    }
  }
  else {
    console.log('La variable no es un número');
  }
}
```

```
let m = [[1,2],[2,3],[3,4]]
sumaTotal = 0;
for (let i = 0; i < m.length; i++) {
  for (let j = 0; j < m[i].length; j++){
    sumaTotal += m[i][j];
  }
}
console.log(sumaTotal);
```

Tractament d' errors mitjançant try.... catch

Quan executem un codi poden aparèixer múltiples errors. A JS existeixen tipus de dades per a ells, que ens ofereix informació addicional sobre l'esdeveniment, com poden ser `TypeError`, `SyntaxError`, `RangeError`,...

Podem manejar-los mitjançant el mandat **try... catch**. El programa intentarà executar el bloc dins de la declaració **try**, si hi ha un error, saltarà al bloc de codi contingut en **catch**.

Mitjançant **finally** definim un bloc de codi que s'executi hi hagi o no error després de l'estructura `try... catch`

```
try {  
    // Bloque a ejecutar  
} catch (error) {  
    // Bloque para tratar un error en el bloque try  
} finally {  
    // Bloque a ejecutar tras try..catch  
};
```

Funcions fletxa

Les funcions fletxa ens permeten escriure funcions d'una manera molt més senzilla i curta.

```
/*Nombre de la función*/ = (/*Parametros*/) => { /* Bloque de código*/ };  
  
getColor = object => { return object.color };  
addOne = (a) => a + 1;  
print = b => {  
    b = b.toString();  
    console.log('Es el numero '.concat(b))  
}
```

```
function addElements(list){  
    for (element of list){  
        sumaTotal += element;  
    }  
    return sumaTotal;  
}
```



```
addElements = (list) => {  
    for (element of list){  
        sumaTotal += element;  
    }  
    return sumaTotal;  
};
```


Classes en JS

Les classes representen una sintaxi molt més clara i simple per crear objectes. Estan compostes per:

- Declaracions de classes: Usades per definir una classe, amb la paraula class i un nom per a ella.
- Constructors: Mètodes especials per crear i inicialitzar un objecte creat amb una classe. Només n'hi pot haver un amb el nom constructor.
- Mètodes de la classe: Diferents mètodes que poden ser aplicats a cada objecte pertanyent a aquesta classe.

```
class Rectangle{  
  constructor (width, height){  
    this.height = height;  
    this.width = width;  
  }  
}
```

```
  getArea() {  
    return this.width * this.height;  
  }  
  hasEqualSides(){  
    if (this.width == this.height){  
      return true;  
    } else {  
      return false;  
    }  
  }  
}
```

Arxius JSON

JSON és un format per transportar i guardar dades. És usat freqüentment a l'hora de rebre dades d'un servidor. Són parells nom-valor, amb les dades separades per comes. Hi ha diferents tipus de dades:

- Nom-valor: Un nom que representa un camp seguit d' un valor.
- Objectes: Encerrados entre claus.
- Arrays: Semblants als arrays en JS, poden contenir objectes, que s' emmagatzemen entre corxets.

Per convertir de JSON a JavaScript utilitzarem la funció JSON.

```
{  
  "nombre": "Juan",  
  "edad": 40,  
  "hobbies": [  
    {"deporte": "futbol"},  
    {"deporte": "baloncesto"}  
  ]  
}
```

Manipular el DOM des de JS

Tenim diverses funcions que ens permeten realitzar això:

- **document.getElementById:** Ens permet seleccionar un element HTML pel seu id per manipular-lo.
- **document.getElementsByTagName:** Ens permet seleccionar elements HTML pel seu nom d'etiqueta per manipular-los.
- **document.getElementsByClassName:** Ens permet seleccionar elements HTML per la seva classe a CSS per manipular-los.
- **element.innerHTML:** Ens permet inserir codi HTML des de JavaScript en un element que seleccionem.
- **element.style.property:** Ens permet canviar el valor d'una propietat d'un estil dins d'un element.
- **element.setAttribute:** Ens permet afegir un nou atribut a un element HTML, amb un parell nom-valor.